

ARGOS

Adaptive Response Guard with Orchestrated Surveillance

Manual del integrante

Enzo Ordoñez Flores

Líder técnico — SOAR · HITL · Notificaciones

Entrega final: 13 de junio de 2026

Universidad de Lima · Tópicos Avanzados de Ciberseguridad · 2026-1

Versión 3.0 — manual organizado por orden de implementación

Parte I — Introducción común al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene el contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Campo	Valor
Tipo	Introducción común para los 4 integrantes
Estado	Activo
Entrega final	13 de junio de 2026 (sábado)
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es parte del curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30 % del peso), demo en vivo (~40 %), y presentación (~20 %). Los seguimientos intermedios pesan el ~10 % restante.

El **activo defendido** es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como

`criticality=production-critical` , lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El **alcance multi-vector** (post ADR-0008) cubre tres categorías de ataques distintas:

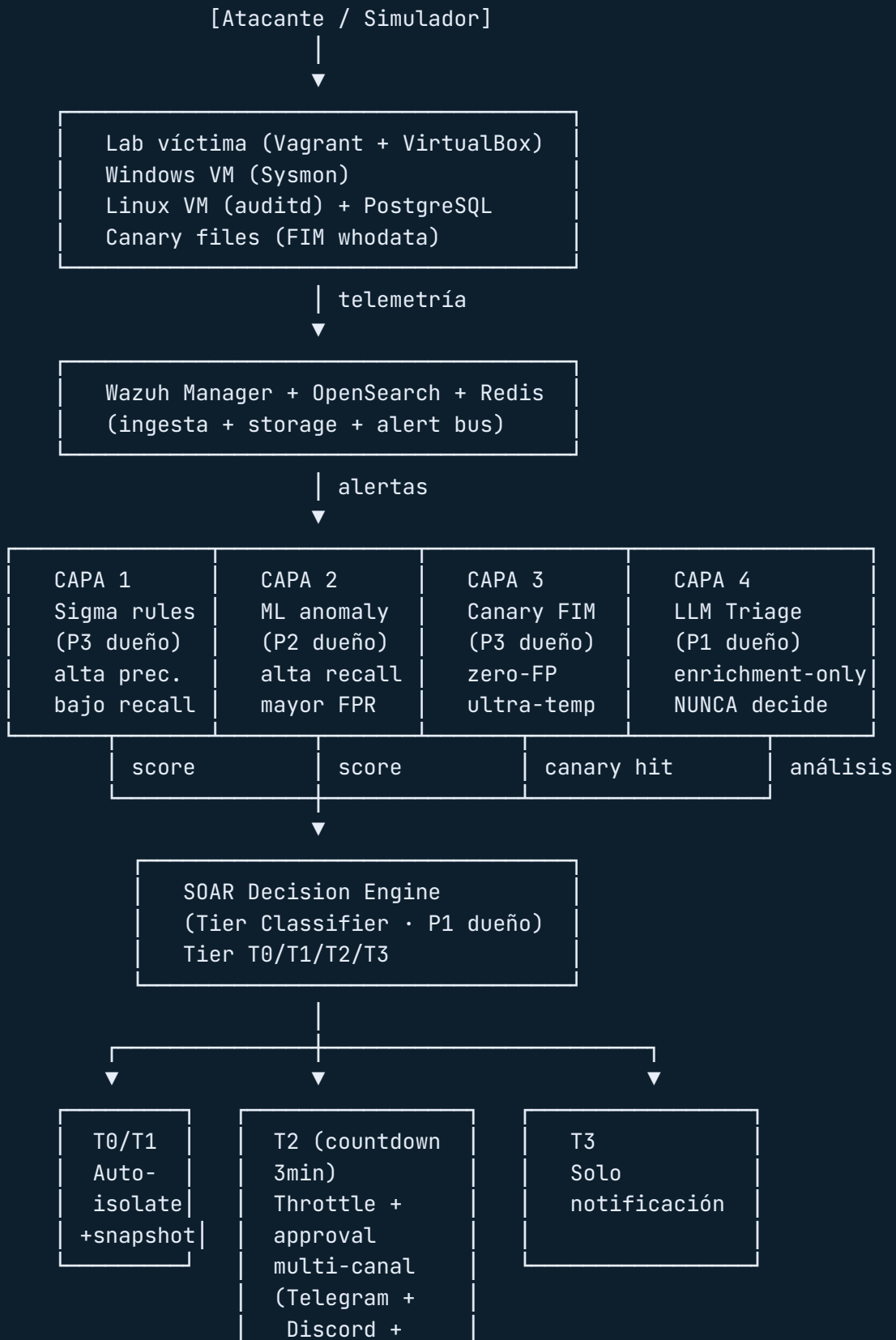
- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest` .
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap` , y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.





Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: `file_write_rate`, `avg_entropy` (Shannon), `extension_modification_ratio`, `crypto_api_calls`, `new_outbound_connections`, `cpu_burst_score`, `io_burst_score`.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: `connections/sec`, `packet rate`, `source IP entropy`, `packet size variance`.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: `rows_returned`, `query_duration_ms`, `hour_of_day`, `user_id`, `query_template_hash`.

Ensemble ransomware: $\text{ensemble_score} = 0.6 \times \text{isolation_forest_score} + 0.4 \times \text{one_class_svm_score}$. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si alucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
 T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
 T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
 T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
 T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md` §Q5.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:



BASH

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim
```

Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.

- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: file_write_rate=347 archivos/min, avg_entropy=7.8 (alta), extension_modification_ratio=0.95, crypto_api_calls=42, cpu_burst_score=0.9.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).
4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el AlertContext con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve TriageResponse: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.

- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block` .
2. **Process kill**: identifica el PID del simulador desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>` .
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED` .

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 chat_ids configurados: mensaje con incident_id, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh` . Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, incident_id, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED` , decisión `EXECUTE_ISOLATION` , política aplicada `auto-execute` , justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: alert created → tier classified → playbook executed → notifications sent.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+		

Categoría	Componente	Función	Owner principal
		Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US-based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+	Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	

Categoría	Componente	Función	Owner principal
			P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- Branch principal: `main`, protegida en GitHub.
- Feature branches: `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- Commits convencionales: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- PRs requieren: CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- Push diario obligatorio antes de las 22:00 cada día de trabajo.

Estructura del repo (post ADR-0008)



```

argos/
├── README.md           # Entry point del proyecto
├── LICENSE             # MIT
├── pyproject.toml      # Metadata + extras por módulo
├── .env.example        # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/   # Pydantic v2 cross-team (entregado · v1.1.0 · 69
│   ├── _mitre_data.py # MITRE_WHITELIST curado
│   ├── alert.py        # WazuhAlert, NormalizedAlert
│   ├── ml_score.py     # MLScore (P2)
│   ├── triage.py       # AlertContext, TriageResponse (P1)
│   ├── incident.py     # Incident state machine (P1)
│   ├── approval.py     # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py        # Tier, Severity, NotificationChannelType, ...
│   └── tests/          # 69 validation tests
├── llm_triage/        # P1 – Capa 4 LLM Triage
│   ├── api/main.py     # FastAPI /triage endpoint
│   ├── llm_client/     # OpenAI primary + Llama local fallback
│   ├── rag/            # BM25 + BGE-large + RRF
│   └── prompts/        # Jinja2 templates
├── soar/              # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/ # tier_classifier, state_machine, orchestrator
│   ├── approval/       # api, jwt_signer, consolidation
│   ├── notification/   # telegram, discord, twilio_voice, email
│   └── playbooks/      # host_isolation, process_kill, snapshot, thrott
├── ml/                # P2 – Capa 2 ML
│   ├── features/       # Feature extractors por dominio
│   ├── models/         # ransomware_ensemble, network_traffic, query_pa
│   └── consumer/       # Redis stream consumer
├── detection/         # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/ # T1486, T1490, T1083, T1562
│   │   ├── network/    # T1498, T1499 rate-based
│   │   ├── database/   # query patterns
│   │   └── webapp/     # T1190 SQLi signatures
│   └── wazuh-rules/    # Convertidas con sigma-cli
├── deception/        # P3 – Capa 3 Canary
│   ├── canary_generator.py # Genera canary files
│   └── fim-configs/      # Wazuh FIM rules

```

```

├── ui/                                # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/            # JSON dashboards exportados
├── attack-simulation/                  # Simuladores multi-vector
│   ├── ransomware_simulator/           # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/                # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/                 # P4 (sqlmap)
│   └── pg_simulator/                   # P4 (SELECT masivo para UC-07)
├── lab/                                # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/                      # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/                         # P2 + P4 – Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md                # 90-second overview
│   ├── CONTEXT.md                     # Onboarding completo
│   ├── PROJECT_STATUS.md              # Honest snapshot
│   ├── EVALUATION_CRITERIA.md         # Rúbrica del curso
│   ├── data-handling.md               # T-030 sanitization policy
│   ├── README.md                      # Mapa de docs
│   ├── architecture/                  # SAD, threat model, flow diagrams
│   ├── decisions/                     # 8 ADRs + OPEN_QUESTIONS
│   ├── use-cases/                     # 8 UCs detallados
│   └── team/                          # Manuales de integrante (este documento + 4 individuos)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual.

Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).

- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde NNN es contador diario zero-padded a 3 dígitos. Ejemplo:

`INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar a implementar. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

 BASH

```
cd ~/projects                # o donde guardes proyectos
git clone https://github.com/Enzo0rdonez/argos.git
cd argos
```

Paso 2 — Crear virtualenv Python

 BASH

```
python3 -m venv .venv
source .venv/bin/activate      # Linux/macOS
# .venv\Scripts\activate      # Windows PowerShell
pip install -U pip setuptools wheel
```

Paso 3 — Instalar dependencias core + tu rol

 BASH

```
# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"      # P1 (siempre + más abajo)
pip install -e ".[ml]"            # P2
pip install -e ".[soar,llm]"      # P1 (servicios)
pip install -e ".[ui]"            # P4
```

Paso 4 — Verificar contracts

 BASH

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

 BASH

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git



```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/setup-inicial
```

11. Comunicación del equipo durante la implementación

Standup diario

- Hora: 9:00 AM
- Canal: Discord `#argos-standup`, llamada de voz
- Duración: 20 minutos (cada integrante ~3-4 min)
- Formato (per `docs/team/standup-template.md`):
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc arquitectónica.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.

2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning queda para después del demo.
3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega para el demo

Para que no te sorprendas cuando llegue el demo sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas).
- **UC-03 split-brain con 4 aprobadores reales** no scripted.
- **UC-05 stealth attack** end-to-end pulido.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos).
- **Video demo final editado.**
- **Informe técnico final.**
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa para el demo: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, más dos rehearsals previos.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual individual	docs/team/manual-p<X>-<nombre>.md
Plan operacional general	docs/team/manual-equipo.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual de Enzo Ordoñez Flores

Manual P1 — Enzo Ordoñez Flores · Líder técnico

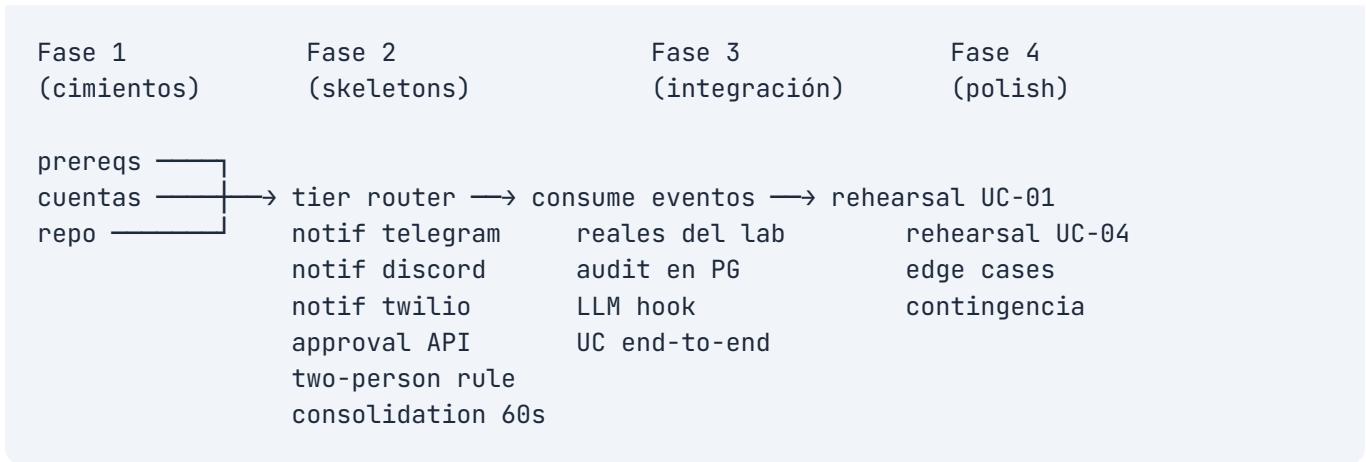
Campo	Valor
Rol	Líder técnico e integrador cross-layer
Owns	SOAR Decision Engine (<code>soar/</code>) · HITL workflow · Notification Service · Approval API · <code>argos_contracts</code>
No owns	Layers 1/3 (P3) · Layer 2 ML + Layer 4 LLM (P2) · Lab + UI + DB (P4)
Outputs blocking	<code>Incident</code> model en Redis → P4 lee la UI · Tier router + Approver workflow → demo UC-04
Entrega final	13 de junio de 2026 (sábado)
Cómo leer	Linealmente. Cada Fase asume la anterior cerrada. Cada sub-sección termina con un bloque de Verificación ; no avances si falla algo.

0. Tu charter en una frase

Tú haces que las 4 capas hablen entre ellas, que un humano pueda aprobar/rechazar acciones críticas, y que las notificaciones lleguen a 4 canales sin caerse. Si tu pieza falla, el demo falla.

0.1 Tu camino crítico

SALIDA ESPERADA



Las Fases son **orden de implementación**, no calendario. Avanzas cuando la anterior pasa su checklist.

0.2 Cómo leer cada sub-sección

Cada componente sigue la misma plantilla:

1. **Contexto** — para qué sirve, por qué importa.
2. **Pasos manuales** — si hay instalación o configuración que NO es un comando (clicks en una web, registros en un servicio externo).
3. **Comandos** — bloque copiable.
4. **Salida esperada** — qué debe imprimir el comando si todo está bien.
5. **Verificación** — comando(s) específicos para confirmar que el componente quedó funcional antes de seguir.
6. **Si algo falla** — síntoma observable → causa → fix.

Fase 1 — Cimientos

1.1 Prerequisites del sistema

Contexto

Confirmas que tu laptop tiene las versiones exactas de Python, git, Docker y herramientas CLI antes de tocar el código. El 80 % de los errores raros viene de versiones desalineadas entre integrantes.

Comandos



BASH

```
python3 --version
pip --version
git --version
docker --version && docker compose version
redis-cli --version
curl --version | head -1
jq --version
```

Salida esperada

SALIDA ESPERADA

```
Python 3.11.7
pip 23.x.x from /usr/lib/python3/dist-packages/pip (python 3.11)
git version 2.34.1
Docker version 24.x.x, build xxxxx
Docker Compose version v2.x.x
redis-cli 7.0.x
curl 7.81.0 (x86_64-pc-linux-gnu) ...
jq-1.6
```

Verificación

VERIFICACIÓN

```
python3 -c "import sys; assert sys.version_info[:2] == (3, 11), sys.version" && echo "Python OK"
docker ps >/dev/null && echo "Docker OK"
redis-cli --version >/dev/null && echo "redis-cli OK"
```

Esperado:

SALIDA ESPERADA

```
Python OK
Docker OK
redis-cli OK
```

Si algo falla

Síntoma	Causa	Fix
<code>python3 --version</code> no es 3.11	Versión equivocada del sistema	Ubuntu: <code>sudo add-apt-repository ppa:deadsnakes/ppa && sudo apt install python3.11 python3.11-venv python3.11-dev</code> macOS: <code>brew install python@3.11</code>
<code>docker ps</code> da <code>Cannot connect to the Docker daemon</code>	Daemon parado o usuario sin grupo <code>docker</code>	<code>sudo systemctl start docker</code> + <code>sudo usermod -aG docker \$USER && newgrp docker</code>
<code>redis-cli</code> not found	Paquete no instalado	<code>sudo apt install redis-tools</code>

1.2 Crear cuentas externas

Contexto

ARGOS depende de cuatro servicios externos para LLM y notificaciones. Las cuatro tienen tier gratuito suficiente para el demo. Las credenciales se guardan en `.env` (que NUNCA se commitea).

1.2.1 Bot de Telegram (canal primario de notificaciones)

Pasos manuales

1. Abre **Telegram** (móvil o desktop).
2. Busca el usuario `@BotFather` y abre su chat.
3. Envía el mensaje `/newbot`.
4. Cuando pida nombre, responde `ARGOS Alerts Bot`.
5. Cuando pida username, responde `argos_alerts_<tu_inicial>_bot` (debe terminar en `_bot`).
6. BotFather responde con un token con formato `7123456789:AAFxxxxxxx...`. **Cópialo** — es tu `TELEGRAM_BOT_TOKEN`.
7. Para obtener tu `TELEGRAM_CHAT_ID`:
 1. Envía cualquier mensaje al bot recién creado (por ejemplo, `hola`).
 2. Abre en navegador `https://api.telegram.org/bot<TU_TOKEN>/getUpdates`.
 3. En la respuesta JSON busca `"chat":{"id": 123456789, ...}`. Ese número es tu `TELEGRAM_CHAT_ID`.

Verificación

VERIFICACIÓN

```
export TELEGRAM_BOT_TOKEN="7123456789:AAFxxxx..." # ← reemplaza
export TELEGRAM_CHAT_ID="123456789" # ← reemplaza
curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage" \
  -d "chat_id=${TELEGRAM_CHAT_ID}" \
  -d "text=ARGOS test desde $(hostname)" | jq .ok
```

Esperado:

SALIDA ESPERADA

true

Y en tu Telegram aparece el mensaje `ARGOS test desde <hostname>`.

Si algo falla

Síntoma	Causa	Fix
<code>"chat not found"</code>	Nunca enviaste mensaje al bot desde tu cuenta	Abre <code>t.me/<usuario_bot></code> , envía <code>/start</code> , vuelve a consultar <code>getUpdates</code>
<code>"Unauthorized"</code>	Token mal copiado	Re-pide el token con <code>/mybots</code> → tu bot → <code>API Token</code>
<code>false</code> sin error visible	<code>chat_id</code> equivocado	Revisa el JSON de <code>getUpdates</code> ; el campo correcto es <code>message.chat.id</code> , no <code>from.id</code>

1.2.2 Webhook de Discord (canal secundario)

Pasos manuales

1. Crea un servidor Discord nuevo y nómbralo `ARGOS Demo`.
2. Crea un canal de texto llamado `#argos-alerts`.
3. Pasa el cursor sobre el canal y click en el engranaje (Editar canal).
4. **Integrations → Webhooks → New Webhook.**
5. Nombre: `ARGOS Notifier`. Avatar opcional.
6. Click **Copy Webhook URL**. Formato: `https://discord.com/api/webhooks/<id>/<token>`. Esa URL es tu `DISCORD_WEBHOOK_URL`.

Verificación


VERIFICACIÓN

```
export DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/.../..."
curl -s -o /dev/null -w "HTTP %{http_code}\n" -X POST "$DISCORD_WEBHOOK_URL" \
-H "Content-Type: application/json" \
-d '{"content": "ARGOS webhook test"}'
```

Esperado:

SALIDA ESPERADA

HTTP 204

(204 = No Content, éxito). El mensaje aparece en `#argos-alerts`.

Si algo falla

HTTP	Causa	Fix
401	Token mal copiado o webhook revocado	Recrea el webhook
404	Webhook eliminado o canal borrado	Recrea el webhook en otro canal

1.2.3 Twilio (escalación T2 por voz DTMF)

Pasos manuales

1. Crea cuenta en `https://www.twilio.com/try-twilio` (trial gratis, USD 15 de crédito).
2. Verifica tu celular como verified caller (Twilio trial sólo llama a números verificados; OK para el demo).
3. **Console** → **Account** → **API keys & tokens**, copia:
 - **Account SID** → `TWILIO_ACCOUNT_SID`.
 - **Auth Token** → `TWILIO_AUTH_TOKEN`.
4. **Phone Numbers** → **Get a trial number** (USA, gratis). El número que te asignan es `TWILIO_FROM_NUMBER`, formato `+1XXXXXXXXXX`.

Verificación

VERIFICACIÓN

```
export TWILIO_ACCOUNT_SID="ACxxxx"
export TWILIO_AUTH_TOKEN="xxxx"
curl -sX GET "https://api.twilio.com/2010-04-01/Accounts/${TWILIO_ACCOUNT_SID}.json"
-u "${TWILIO_ACCOUNT_SID}:${TWILIO_AUTH_TOKEN}" | jq .status
```

Esperado:

SALIDA ESPERADA

"active"

Si algo falla

Síntoma	Causa	Fix
"unauthorized"	SID o token mal copiados	Revisa API keys & tokens y vuelve a copiar
El día del demo no entra la llamada al Perú	Restricción geográfica del trial	Plan B documentado en ADR-0007 v2: escalación T2 cae a Telegram + Discord con prefijo [T2-ESCALATION]

1.2.4 OpenAI API key (LLM Triage Layer 4)

Pasos manuales

1. Entra a <https://platform.openai.com/api-keys>.
2. Click **Create new secret key**. Nombre: **ARGOS demo**.
3. Copia la key (empieza con **sk-proj- ...**) → **OPENAI_API_KEY**.
4. Set un límite de billing de USD 5/mes en <https://platform.openai.com/account/billing/limits> para protegerte.

Verificación

 VERIFICACIÓN

```
export OPENAI_API_KEY="sk-proj-xxxx"
curl -s https://api.openai.com/v1/models \
  -H "Authorization: Bearer ${OPENAI_API_KEY}" | jq '.data[0].id'
```

Esperado:

SALIDA ESPERADA

```
"gpt-4o-mini-2024-07-18"
```

(O cualquier ID de modelo; lo que **no** debe aparecer es `"invalid_api_key"` o `"insufficient_quota"` .)

1.3 Clonar el repo y preparar el entorno Python

Contexto

Creas tu working tree, virtualenv aislado y dependencias. El virtualenv es **obligatorio**: instalar paquetes en el Python del sistema rompe otras cosas.

Comandos

 BASH

```
mkdir -p ~/code && cd ~/code
git clone git@github.com:Enzo0rdonez/argos.git
cd argos

git status

python3 -m venv .venv
source .venv/bin/activate

pip install --upgrade pip
pip install -e ./argos_contracts
pip install -r soar/requirements.txt
```

Salida esperada

SALIDA ESPERADA

```
Cloning into 'argos'...
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
Successfully installed pip-24.x.x
Successfully installed argos_contracts-1.1.0 pydantic-2.x.x ...
Successfully installed fastapi-0.110.x uvicorn-0.27.x redis-5.0.x httpx-0.27.x ...
```

Verificación

VERIFICACIÓN

```
which python && python -c "import argos_contracts; print('argos_contracts', argos_co
pytest -q 2>&1 | tail -3
```

Esperado:

SALIDA ESPERADA

```
/home/<usuario>/code/argos/.venv/bin/python
argos_contracts 1.1.0
69 passed, 1 warning in 0.27s
```

Si algo falla

Síntoma	Causa	Fix
<code>ModuleNotFoundError: argos_contracts</code> después de <code>pip install -e</code>	Instalaste fuera del venv	<code>which python</code> debe terminar en <code>.venv/bin/python</code> . Si no, <code>source .venv/bin/activate</code>
<code>pytest: command not found</code>	Falta dep del SOAR	<code>pip install -r soar/requirements.txt</code>
<code>git clone</code> pide password HTTPS	No tienes SSH key configurada	<code>gh auth login</code> o <code>git clone https://github.com/Enzo0rdonez/argos.git</code>

1.4 Crear tu `.env`

Contexto

`.env` vive en la raíz del repo y se carga con `python-dotenv` o `export $(grep -v '^#' .env | xargs)`. Está en `.gitignore`, así que tus credenciales no se filtran.

Pasos manuales

1. Copia los valores que recolectaste en 1.2 (4 servicios).
2. Genera el archivo con el comando de abajo.
3. **No commitees** `.env`. Verifica con `git status` que no aparezca.

Comandos

 BASH

```
cat > .env << 'EOF'
# === Telegram ===
TELEGRAM_BOT_TOKEN=7123456789:AAFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TELEGRAM_CHAT_ID=123456789

# === Discord ===
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/.../...

# === Twilio ===
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_FROM_NUMBER=+1XXXXXXXXXX
TWILIO_TO_NUMBER=+51XXXXXXXXXX

# === OpenAI ===
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
LLM_BACKEND=openai_gpt4o_mini

# === Redis (lab Vagrant local) ===
REDIS_URL=redis://localhost:6379/0

# === PostgreSQL (audit log) ===
ARGOS_PG_URL=postgresql://argos:argos@localhost:5432/argos_audit
EOF
chmod 600 .env
```

(Reemplaza los placeholders con tus valores reales.)

Verificación

 VERIFICACIÓN

```
ls -la .env
git check-ignore .env
test -s .env && echo "env NO vacío"
```

Esperado:

SALIDA ESPERADA

```
-rw----- 1 usuario usuario 614 may 24 18:00 .env
.env
env NO vacío
```

(`git check-ignore .env` imprime el nombre si el archivo está en `.gitignore` ; si no imprime nada → falta entrada en `.gitignore`).

Checklist Fase 1

#	Check	OK
1	Python 3.11.x, Docker, redis-cli, jq presentes	☐
2	Bot Telegram envía mensaje real	☐
3	Webhook Discord devuelve HTTP 204	☐
4	Twilio cuenta <code>"status": "active"</code>	☐
5	OpenAI key responde a <code>/v1/models</code>	☐
6	<code>pytest -q</code> → <code>69 passed</code>	☐
7	<code>.env</code> creado y <code>git check-ignore</code> lo detecta	☐

No avances a Fase 2 si algún check está rojo.

Fase 2 — Skeletons funcionales

Goal: cada componente individual corre y pasa sus tests unitarios, **aún con stubs** en lugar de servicios reales. La conexión al lab real viene en Fase 3.

2.1 SOAR Decision Engine — Tier Router

Contexto

El Tier Router es la pieza más crítica de todo ARGOS: recibe un evento normalizado de cualquier capa y decide qué Tier asignar (T0/T1/T2/T3). De ese tier salen las acciones automáticas y/o la solicitud de aprobación humana.

Estructura

SALIDA ESPERADA

```
soar/
├── requirements.txt
├── decision_engine/
│   ├── __init__.py
│   ├── tier_router.py    ← tú escribes
│   ├── policies.py      ← tú escribes (matrices de tier)
│   └── tests/
│       ├── test_tier_router.py
│       └── test_policies.py
```

policies.py



```

"""Matrices y constantes de política para el Tier Router.

Cualquier cambio aquí cambia el comportamiento del demo. Una sola fuente de verdad
evita branchear lógica en múltiples archivos.
"""

from argos_contracts.enums import Severity, Tier

# Técnicas que disparan T0 automático.
AUTO_T0_TECHNIQUES: frozenset[str] = frozenset({
    "T1486", # Data Encrypted for Impact
    "T1490", # Inhibit System Recovery
    "T1485", # Data Destruction
    "T1561", # Disk Wipe
})

# A partir de cuántas capas coincidentes consideramos "consenso" → boost a T0.
MIN_LAYERS_FOR_AUTO: int = 3

# Mapeo base severity → Tier antes de aplicar boosts.
SEVERITY_TO_BASE_TIER: dict[Severity, Tier] = {
    Severity.CRITICAL: Tier.T0,
    Severity.HIGH:     Tier.T1,
    Severity.MEDIUM:   Tier.T2,
    Severity.LOW:      Tier.T3,
}

```

`tier_router.py`



```

"""Tier Router – asigna T0/T1/T2/T3 a un evento normalizado."""

from __future__ import annotations
from typing import Literal

from argos_contracts.enums import Tier
from argos_contracts.incident import NormalizedEvent
from soar.decision_engine.policies import (
    AUTO_T0_TECHNIQUES, MIN_LAYERS_FOR_AUTO, SEVERITY_T0_BASE_TIER,
)

ConfidenceBand = Literal["high", "medium", "low"]

def confidence_band(score: float) → ConfidenceBand:
    if score ≥ 0.85: return "high"
    if score ≥ 0.55: return "medium"
    return "low"

def route(event: NormalizedEvent) → Tier:
    """Devuelve el Tier asignado a un evento. Pura: no I/O."""
    # 1. Auto-T0 por técnica MITRE crítica
    if event.mitre_technique in AUTO_T0_TECHNIQUES:
        return Tier.T0

    # 2. Tier base por severity
    base = SEVERITY_T0_BASE_TIER[event.severity]

    # 3. Boost por número de capas
    if event.num_layers_fired ≥ MIN_LAYERS_FOR_AUTO:
        if base in (Tier.T1, Tier.T2):
            return Tier.T0

    # 4. Down-tier si confianza baja (single layer + low confidence)
    band = confidence_band(event.confidence_score)
    if event.num_layers_fired == 1 and band == "low":
        return Tier.T3

    return base

```

Tests



```

# soar/decision_engine/tests/test_tier_router.py
import pytest

from argos_contracts.enums import Severity, Tier
from argos_contracts.incident import NormalizedEvent
from soar.decision_engine.tier_router import route, confidence_band

def make_event(**overrides) → NormalizedEvent:
    base = dict(
        event_id="evt-test-001", severity=Severity.MEDIUM,
        mitre_technique="T1083", num_layers_fired=1,
        confidence_score=0.7, host="WIN-VICTIM-01", layer_origin="sigma",
    )
    base.update(overrides)
    return NormalizedEvent(**base)

@pytest.mark.parametrize("score, expected", [
    (0.95, "high"), (0.85, "high"),
    (0.84, "medium"), (0.55, "medium"),
    (0.54, "low"), (0.10, "low"),
])
def test_confidence_band(score, expected):
    assert confidence_band(score) == expected

def test_auto_t0_for_ransomware_technique():
    assert route(make_event(mitre_technique="T1486", severity=Severity.LOW)) == Tier.T0

def test_critical_maps_to_t0():
    assert route(make_event(severity=Severity.CRITICAL)) == Tier.T0

def test_three_layers_boost_t2_to_t0():
    assert route(make_event(num_layers_fired=3)) == Tier.T0

def test_single_layer_low_confidence_drops_to_t3():
    assert route(make_event(confidence_score=0.3)) == Tier.T3

```

```
def test_medium_two_layers_stays_t2():
    assert route(make_event(num_layers_fired=2, confidence_score=0.7)) == Tier.T2
```

Comandos



BASH

```
pytest soar/decision_engine/tests/test_tier_router.py -v
```

Salida esperada

SALIDA ESPERADA

```
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.95-high] PASSED
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.85-high] PASSED
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.84-medium] PASSED
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.55-medium] PASSED
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.54-low] PASSED
soar/decision_engine/tests/test_tier_router.py::test_confidence_band[0.1-low] PASSED
soar/decision_engine/tests/test_tier_router.py::test_auto_t0_for_ransomware_technique PAS
soar/decision_engine/tests/test_tier_router.py::test_critical_maps_to_t0 PASSED
soar/decision_engine/tests/test_tier_router.py::test_three_layers_boost_t2_to_t0 PASSED
soar/decision_engine/tests/test_tier_router.py::test_single_layer_low_confidence_drops_to
soar/decision_engine/tests/test_tier_router.py::test_medium_two_layers_stays_t2 PASSED
===== 11 passed in 0.08s =====
```

Verificación



VERIFICACIÓN

```
python -c "from soar.decision_engine.tier_router import route; print('import OK')"
pytest --cov=soar.decision_engine.tier_router soar/decision_engine/tests/ 2>&1 | gro
```

Esperado:

SALIDA ESPERADA

```
import OK
soar/decision_engine/tier_router.py    16      0    100%
```

Si algo falla

Síntoma	Causa	Fix
<code>ImportError: cannot import name 'Tier'</code>	<code>argos_contracts</code> desactualizado	<code>pip install -e ./argos_contracts --force-reinstall</code>
Tests <code>test_three_layers_boost_t2_to_t0</code> falla	Lógica de boost mal aplicada	Verifica que el <code>if base in (Tier.T1, Tier.T2)</code> use <code>in</code> , no <code>=</code>
Coverage < 100 %	Falta caso edge	Agrega test para <code>severity=HIGH + num_layers=1 + confidence=0.4</code> (espera T1, no T3)

2.2 Notification Service — Estructura base

Contexto

Adapter pattern: una interfaz `NotificationChannel` con N implementaciones (Telegram, Discord, Twilio, Email). El `NotificationService` recibe un `Incident` y despacha a los canales correspondientes según política. Cada canal nunca tira excepción hacia afuera; devuelve un `DispatchResult` que el servicio puede degradar.

base.py



```
"""Interfaz común a todos los canales de notificación."""

from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

from argos_contracts.enums import NotificationChannelType
from argos_contracts.incident import Incident

@dataclass(frozen=True)
class DispatchResult:
    channel: NotificationChannelType
    success: bool
    latency_ms: int
    error: Optional[str] = None

class NotificationChannel(ABC):
    channel_type: NotificationChannelType # subclase setea como class var

    @abstractmethod
    def dispatch(self, incident: Incident) → DispatchResult:
        """Envía notificación. Nunca tira excepción al caller."""
        ...
```

service.py




```

"""Orquesta despacho a múltiples canales según el Tier del incidente."""

from __future__ import annotations
import logging, time
from typing import Iterable

from argos_contracts.enums import Tier, NotificationChannelType
from argos_contracts.incident import Incident
from soar.notifications.base import NotificationChannel, DispatchResult

logger = logging.getLogger(__name__)

TIER_CHANNELS: dict[Tier, list[NotificationChannelType]] = {
    Tier.T0: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T1: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T2: [NotificationChannelType.TELEGRAM, NotificationChannelType.DISCORD],
    Tier.T3: [],
}

class NotificationService:
    def __init__(self, channels: Iterable[NotificationChannel]):
        self._channels = {c.channel_type: c for c in channels}

    def dispatch_for_tier(self, incident: Incident) → list[DispatchResult]:
        wanted = TIER_CHANNELS.get(incident.tier, [])
        results: list[DispatchResult] = []
        for channel_type in wanted:
            channel = self._channels.get(channel_type)
            if channel is None:
                results.append(DispatchResult(
                    channel=channel_type, success=False,
                    latency_ms=0, error="channel not configured",
                ))
                continue
            t0 = time.monotonic()
            try:
                results.append(channel.dispatch(incident))
            except Exception as exc: # noqa: BLE001
                logger.exception("channel %s raised", channel_type)
                results.append(DispatchResult(
                    channel=channel_type, success=False,
                    latency_ms=int((time.monotonic() - t0) * 1000),
                ))

```

```

        error=f"unexpected: {type(exc).__name__}: {exc}",
    ))
    return results

def escalate_to_voice(self, incident: Incident) → DispatchResult:
    voice = self._channels.get(NotificationChannelType.TWILIO_VOICE)
    if voice is None:
        return DispatchResult(
            channel=NotificationChannelType.TWILIO_VOICE,
            success=False, latency_ms=0, error="twilio not configured",
        )
    return voice.dispatch(incident)

```

Verificación

VERIFICACIÓN

```

python -c "from soar.notifications.service import NotificationService; print('service import OK')"
pytest soar/notifications/tests/test_service.py -q

```

Esperado:

SALIDA ESPERADA

```

service import OK
5 passed in 0.04s

```

2.3 Notification Channel — Telegram

Contexto

Mensajes con formato MarkdownV2 (Telegram-flavored). Botones inline `[Approve][Reject]` cuando `incident.requires_approval = True`.

`channels/telegram.py`



PYTHON

```

"""Telegram bot – canal primario para todos los tiers ≠ T3."""

from __future__ import annotations
import logging, os, time
from typing import Optional
import httpx

from argos_contracts.enums import NotificationChannelType, Tier
from argos_contracts.incident import Incident
from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

_API = "https://api.telegram.org/bot{token}/sendMessage"

def _escape_md(text: str) → str:
    for ch in r"_*[]()~`>#+-=|{}.\!\\":
        text = text.replace(ch, f"\{ch}")
    return text

def _format(incident: Incident) → str:
    tier_emoji = {Tier.T0: "🅐", Tier.T1: "🅑", Tier.T2: "🅒", Tier.T3: "🅓"}
    return (
        f"{tier_emoji[incident.tier]} *ARGOS {incident.tier.value}* – "
        f"`{_escape_md(incident.host.hostname)}`\n"
        f"*Técnica:* `{_escape_md(incident.mitre_technique)}`\n"
        f"*Capas firing:* `{incident.num_layers_fired}`\n"
        f"*Confianza:* `{incident.confidence_score:.2f}`\n"
        f"*ID:* `{_escape_md(incident.incident_id)} `"
    )

def _inline_keyboard(incident_id: str) → dict:
    return {"inline_keyboard": [[
        {"text": "👍 Approve", "callback_data": f"approve:{incident_id}"},
        {"text": "👎 Reject", "callback_data": f"reject:{incident_id}"},
    ]]}

class TelegramChannel(NotificationChannel):
    channel_type = NotificationChannelType.TELEGRAM

```

```

def __init__(self, bot_token: Optional[str] = None,
              chat_id: Optional[str] = None,
              client: Optional[httpx.Client] = None,
              timeout: float = 5.0):
    self._token = bot_token or os.environ["TELEGRAM_BOT_TOKEN"]
    self._chat_id = chat_id or os.environ["TELEGRAM_CHAT_ID"]
    self._client = client or httpx.Client(timeout=timeout)

def dispatch(self, incident: Incident) → DispatchResult:
    t0 = time.monotonic()
    body = {
        "chat_id": self._chat_id,
        "text": _format(incident),
        "parse_mode": "MarkdownV2",
    }
    if incident.requires_approval:
        body["reply_markup"] = _inline_keyboard(incident.incident_id)
    try:
        r = self._client.post(_API.format(token=self._token), json=body)
        r.raise_for_status()
        payload = r.json()
        if not payload.get("ok"):
            return DispatchResult(
                channel=self.channel_type, success=False,
                latency_ms=int((time.monotonic() - t0) * 1000),
                error=str(payload.get("description")),
            )
        return DispatchResult(
            channel=self.channel_type, success=True,
            latency_ms=int((time.monotonic() - t0) * 1000),
        )
    except httpx.HTTPError as exc:
        return DispatchResult(
            channel=self.channel_type, success=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            error=f"http: {exc}",
        )

```

Smoke test manual



```

export $(grep -v '^#' .env | xargs)

python - << 'PY'
from datetime import datetime, timezone
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.telegram import TelegramChannel

incident = Incident(
    incident_id="inc-smoke-001", tier=Tier.T0, state=IncidentState.NEW,
    severity=Severity.CRITICAL,
    host=HostInfo(hostname="WIN-VICTIM-01", ip="192.168.56.20", os="Windows 10"),
    mitre_technique="T1486", num_layers_fired=3, confidence_score=0.94,
    created_at=datetime.now(timezone.utc), requires_approval=False,
    approvers=[], final_decision=None, consolidation_window=None,
)
print(TelegramChannel().dispatch(incident))
PY

```

Salida esperada

SALIDA ESPERADA

```
DispatchResult(channel=<NotificationChannelType.TELEGRAM: 'telegram'>, success=True, late
```

Y en Telegram aparece:

SALIDA ESPERADA

```

🔊 ARGOS T0 – WIN-VICTIM-01
Técnica: T1486
Capas firing: 3
Confianza: 0.94
ID: inc-smoke-001

```

Verificación



VERIFICACIÓN

```
python -c "from soar.notifications.channels.telegram import TelegramChannel; print(
```

Esperado:

SALIDA ESPERADA

```
telegram OK
```

2.4 Notification Channel — Discord

Contexto

Webhook simple. Mensaje con embed coloreado por tier (T0 rojo, T1 naranja, T2 amarillo, T3 azul). No requiere bot ni token de usuario.

`channels/discord.py`



PYTHON

```

"""Discord webhook channel."""

from __future__ import annotations
import logging, os, time
from typing import Optional
import httpx

from argos_contracts.enums import NotificationChannelType, Tier
from argos_contracts.incident import Incident
from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

_COLOR = {Tier.T0: 0xE53935, Tier.T1: 0xFB8C00, Tier.T2: 0xFDD835, Tier.T3: 0x1E88E5}

def _embed(incident: Incident) → dict:
    return {
        "title": f"ARGOS {incident.tier.value} - {incident.host.hostname}",
        "color": _COLOR[incident.tier],
        "fields": [
            {"name": "Técnica MITRE", "value": incident.mitre_technique, "inline": True},
            {"name": "Capas firing", "value": str(incident.num_layers_fired), "inline": True},
            {"name": "Confianza", "value": f"{incident.confidence_score:.2f}", "inline": True},
            {"name": "Incident ID", "value": f"`{incident.incident_id}`", "inline": False},
        ],
        "footer": {"text": "ARGOS · respond on Telegram for approval"},
    }

class DiscordChannel(NotificationChannel):
    channel_type = NotificationChannelType.DISCORD

    def __init__(self, webhook_url: Optional[str] = None,
                 client: Optional[httpx.Client] = None,
                 timeout: float = 5.0):
        self._url = webhook_url or os.environ["DISCORD_WEBHOOK_URL"]
        self._client = client or httpx.Client(timeout=timeout)

    def dispatch(self, incident: Incident) → DispatchResult:
        t0 = time.monotonic()
        body = {"embeds": [_embed(incident)]}
        try:
            r = self._client.post(self._url, json=body)

```



```

        if r.status_code in (200, 204):
            return DispatchResult(
                channel=self.channel_type, success=True,
                latency_ms=int((time.monotonic() - t0) * 1000),
            )
        return DispatchResult(
            channel=self.channel_type, success=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            error=f"http {r.status_code}: {r.text[:200]}",
        )
    except httpx.HTTPError as exc:
        return DispatchResult(
            channel=self.channel_type, success=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            error=f"http: {exc}",
        )

```

Verificación

VERIFICACIÓN

```

python - << 'PY'
import os
from datetime import datetime, timezone
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.discord import DiscordChannel

i = Incident(
    incident_id="inc-smoke-002", tier=Tier.T2, state=IncidentState.NEW,
    severity=Severity.MEDIUM,
    host=HostInfo(hostname="LIN-VICTIM-01", ip="192.168.56.21", os="Ubuntu 22.04"),
    mitre_technique="T1083", num_layers_fired=2, confidence_score=0.71,
    created_at=datetime.now(timezone.utc), requires_approval=True,
    approvers=[], final_decision=None, consolidation_window=None,
)
r = DiscordChannel().dispatch(i)
print(r.success, r.latency_ms, "ms")
PY

```

Esperado:

```
SALIDA ESPERADA
```

```
True 312 ms
```

Y un embed amarillo en `#argos-alerts`.

2.5 Notification Channel — Twilio Voice (escalación T2)

Contexto

Sólo se dispara para T2 a los 60 segundos sin respuesta. La llamada lee TwiML que enuncia el incidente y captura DTMF (`1=approve` , `2=reject`). Es el último recurso antes de que el conservative-wins policy decida solo.

```
channels/twilio_voice.py
```



PYTHON

```

"""Twilio voice – escalación DTMF."""

from __future__ import annotations
import logging, os, time
from typing import Optional
from urllib.parse import urlencode
import httpx

from argos_contracts.enums import NotificationChannelType
from argos_contracts.incident import Incident
from soar.notifications.base import DispatchResult, NotificationChannel

logger = logging.getLogger(__name__)

def _twiml_url(incident_id: str, base: str) → str:
    return f"{base}/voice/twiml?{urlencode({'incident': incident_id})}"

class TwilioVoiceChannel(NotificationChannel):
    channel_type = NotificationChannelType.TWILIO_VOICE

    def __init__(self,
        account_sid: Optional[str] = None,
        auth_token: Optional[str] = None,
        from_number: Optional[str] = None,
        to_number: Optional[str] = None,
        public_base_url: Optional[str] = None,
        client: Optional[httpx.Client] = None,
        timeout: float = 8.0):
        self._sid = account_sid or os.environ["TWILIO_ACCOUNT_SID"]
        self._tok = auth_token or os.environ["TWILIO_AUTH_TOKEN"]
        self._from = from_number or os.environ["TWILIO_FROM_NUMBER"]
        self._to = to_number or os.environ["TWILIO_TO_NUMBER"]
        self._base = public_base_url or os.environ.get("ARGOS_PUBLIC_URL", "")
        self._client = client or httpx.Client(timeout=timeout, auth=(self._sid, self._tok))

    def dispatch(self, incident: Incident) → DispatchResult:
        t0 = time.monotonic()
        url = f"https://api.twilio.com/2010-04-01/Accounts/{self._sid}/Calls.json"
        try:
            r = self._client.post(url, data={
                "From": self._from, "To": self._to,
                "Url": _twiml_url(incident.incident_id, self._base),
            })

```

```

        "Method": "POST", "Timeout": "20",
    })
    if r.status_code in (200, 201):
        return DispatchResult(
            channel=self.channel_type, success=True,
            latency_ms=int((time.monotonic() - t0) * 1000),
        )
    return DispatchResult(
        channel=self.channel_type, success=False,
        latency_ms=int((time.monotonic() - t0) * 1000),
        error=f"http {r.status_code}: {r.text[:300]}",
    )
except httpx.HTTPError as exc:
    return DispatchResult(
        channel=self.channel_type, success=False,
        latency_ms=int((time.monotonic() - t0) * 1000),
        error=f"http: {exc}",
    )

```

Pasos manuales para que Twilio pueda alcanzar tu Approval API local

Twilio necesita una URL pública para hacer callback al TwiML. Usa **ngrok** :

1. Instala ngrok: <https://ngrok.com/download> .
2. Regístrate gratis en ngrok.com y copia tu authtoken.
3. Configura: `ngrok config add-authtoken <tu_token>` .
4. En una terminal, mantén corriendo: `ngrok http 8001` (el puerto donde corre tu Approval API).
5. ngrok te muestra una URL como `https://abcd-1234.ngrok-free.app` . Exporta:

 BASH

```
export ARGOS_PUBLIC_URL=https://abcd-1234.ngrok-free.app
```

Verificación (gasta ~USD 0.013 de tu trial credit)

 VERIFICACIÓN

```

export ARGOS_PUBLIC_URL=https://abcd-1234.ngrok-free.app

python - << 'PY'
from datetime import datetime, timezone
from argos_contracts.incident import Incident, HostInfo
from argos_contracts.enums import Tier, Severity, IncidentState
from soar.notifications.channels.twilio_voice import TwilioVoiceChannel

i = Incident(
    incident_id="inc-smoke-003", tier=Tier.T2, state=IncidentState.NEW,
    severity=Severity.MEDIUM,
    host=HostInfo(hostname="WIN-VICTIM-01", ip="192.168.56.20", os="Windows 10"),
    mitre_technique="T1083", num_layers_fired=2, confidence_score=0.72,
    created_at=datetime.now(timezone.utc), requires_approval=True,
    approvers=[], final_decision=None, consolidation_window=None,
)
r = TwilioVoiceChannel().dispatch(i)
print(r.success, r.error or "no error")
PY

```

Esperado:

SALIDA ESPERADA

True no error

Y tu celular suena con una llamada de tu número Twilio.

Si algo falla

Síntoma	Causa	Fix
Error <code>21219</code>	Número <code>TWILIO_TO_NUMBER</code> no verificado	Verifícalo en <code>https://console.twilio.com/us1/develop/phone-numbers/manage/verified</code>
Error <code>21401</code>	Número <code>TO</code> mal formateado	Debe empezar con <code>+</code> y código país: <code>+51XXXXXXXXXX</code>
Llamada cae al instante	Twiml URL inalcanzable (ngrok caído)	Vuelve a levantar <code>ngrok http 8001</code> y actualiza <code>ARGOS_PUBLIC_URL</code>

2.6 Approval API — endpoints FastAPI

Contexto

Mini API HTTP que recibe respuestas de aprobación de tres orígenes: callback de botón Telegram, reacción Discord (opcional), y DTMF Twilio. Cada respuesta muta el `Incident` en Redis y dispara la evaluación de la regla two-person.

`main.py`



```

"""Approval API – recibe respuestas y muta Incident en Redis."""

from __future__ import annotations
import os
from contextlib import asynccontextmanager
from typing import AsyncIterator

import redis.asyncio as redis
from fastapi import FastAPI, Form, HTTPException, Response

from soar.approval_api.handlers import (
    record_approval_response, build_final_decision_if_ready,
)
from soar.approval_api.twiml import build_voice_gather_xml, dtmf_to_response

@asynccontextmanager
async def lifespan(app: FastAPI) → AsyncIterator[None]:
    app.state.redis = redis.from_url(
        os.environ["REDIS_URL"], decode_responses=True
    )
    try:
        yield
    finally:
        await app.state.redis.close()

app = FastAPI(title="ARGOS Approval API", lifespan=lifespan)

@app.get("/healthz")
async def healthz() → dict:
    pong = await app.state.redis.ping()
    return {"ok": True, "redis": pong}

@app.post("/telegram/callback")
async def telegram_callback(update: dict) → dict:
    cq = update.get("callback_query")
    if cq is None:
        raise HTTPException(400, "missing callback_query")
    action, _, incident_id = (cq.get("data") or "").partition(":")
    if action not in ("approve", "reject") or not incident_id:
        raise HTTPException(400, "bad callback_data")

```

```

user = cq.get("from", {})
approver_id = f"telegram:{user.get('id')}"
await record_approval_response(
    redis_client=app.state.redis,
    incident_id=incident_id, approver_id=approver_id,
    decision=action, channel="telegram",
)
await build_final_decision_if_ready(app.state.redis, incident_id)
return {"ok": True}

@app.post("/voice/twiml", response_class=Response)
async def voice_twiml(incident: str) → Response:
    return Response(content=build_voice_gather_xml(incident_id=incident),
                    media_type="application/xml")

@app.post("/voice/dtmf", response_class=Response)
async def voice_dtmf(Digits: str = Form(...), incident: str = Form(...)) → Response:
    decision = dtmf_to_response(Digits)
    if decision is None:
        return Response(
            content="<Response><Say>Invalid input. Bye.</Say><Hangup/></Response>",
            media_type="application/xml",
        )
    await record_approval_response(
        redis_client=app.state.redis,
        incident_id=incident, approver_id=f"twilio:{incident}",
        decision=decision, channel="twilio_voice",
    )
    await build_final_decision_if_ready(app.state.redis, incident)
    return Response(
        content=f"<Response><Say>{decision} recorded. Goodbye.</Say><Hangup/></Response>",
        media_type="application/xml",
    )

```

twiml.py




```

"""Helpers para generar TwiML y traducir DTMF."""

from typing import Optional, Literal

Decision = Literal["approve", "reject"]

def build_voice_gather_xml(incident_id: str) → str:
    return f"""<?xml version="1.0" encoding="UTF-8"?>
<Response>
  <Gather numDigits="1" action="/voice/dtmf" method="POST" timeout="20">
    <Say voice="alice">
      ARGOS critical incident. Incident {incident_id}.
      Press one to approve, two to reject.
    </Say>
    <Pause length="2"/>
  </Gather>
  <Say>No input received. Goodbye.</Say>
  <Hangup/>
</Response>"""

_DTMF: dict[str, Decision] = {"1": "approve", "2": "reject"}

def dtmf_to_response(digits: str) → Optional[Decision]:
    return _DTMF.get(digits.strip()) if digits else None

```

Comandos

```


BASH

```

```
uvicorn soar.approval_api.main:app --host 0.0.0.0 --port 8001 --reload
```

Salida esperada (terminal con uvicorn)

```
SALIDA ESPERADA
```

```
INFO: Will watch for changes in these directories: ['/home/usuario/code/argos']
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
INFO: Started reloader process [12345] using StatReload
INFO: Started server process [12346]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Verificación


VERIFICACIÓN

```
curl -s http://localhost:8001/healthz | jq
```

Esperado:

SALIDA ESPERADA

```
{
  "ok": true,
  "redis": true
}
```

Y para el endpoint TwiML:


VERIFICACIÓN

```
curl -s -X POST 'http://localhost:8001/voice/twiml?incident=inc-test' | head -3
```

Esperado:

SALIDA ESPERADA

```
<?xml version="1.0" encoding="UTF-8"?>
<Response>
  <Gather numDigits="1" action="/voice/dtmf" method="POST" timeout="20">
```

2.7 Two-person rule + conservative-wins policy

Contexto

Núcleo del HITL. Cuando un T2 dispara, se notifican N aprobadores (≥ 2). El sistema espera hasta 2 respuestas concordantes o cierra la ventana en 60 s. La política **conservative-wins** dice: ante conflicto, prevalece reject.

handlers.py (lógica completa)



```

"""Lógica HITL: registrar respuestas, evaluar quorum, resolver conflictos."""

from __future__ import annotations
import logging, time
from typing import Literal

import redis.asyncio as redis

from argos_contracts.enums import ApproverStatus, IncidentState
from argos_contracts.incident import ApproverState, FinalDecision, Incident

logger = logging.getLogger(__name__)

QUORUM_NEEDED = 2
DEFAULT_WINDOW_SECONDS = 60

async def _load_incident(r: redis.Redis, incident_id: str) → Incident:
    raw = await r.get(f"incident:{incident_id}")
    if raw is None:
        raise KeyError(f"incident {incident_id} not in Redis")
    return Incident.model_validate_json(raw)

async def _save_incident(r: redis.Redis, incident: Incident) → None:
    await r.set(f"incident:{incident.incident_id}", incident.model_dump_json())

async def record_approval_response(
    *, redis_client: redis.Redis,
    incident_id: str, approver_id: str,
    decision: Literal["approve", "reject"], channel: str,
) → None:
    incident = await _load_incident(redis_client, incident_id)
    if incident.final_decision is not None:
        logger.info("incident %s already decided; ignoring late %s from %s",
                    incident_id, decision, approver_id)
        return

    status = (ApproverStatus.APPROVED if decision == "approve"
              else ApproverStatus.REJECTED)
    now = time.time()

    for ap in incident.approvers:

```

```

        if ap.approver_id == approver_id:
            ap.status = status
            ap.responded_at = now
            ap.channel = channel
            break
    else:
        incident.approvers.append(ApproverState(
            approver_id=approver_id, status=status, channel=channel,
            notified_at=now, responded_at=now,
        ))
    await _save_incident(redis_client, incident)

def _evaluate(incident: Incident) → FinalDecision | None:
    """Pure: dada la lista de approvers, ¿hay decisión?"""
    approved = sum(a.status == ApproverStatus.APPROVED for a in incident.approvers)
    rejected = sum(a.status == ApproverStatus.REJECTED for a in incident.approvers)
    timeout = sum(a.status == ApproverStatus.TIMEOUT for a in incident.approvers)

    if approved ≥ QUORUM_NEEDED and rejected == 0:
        return FinalDecision(
            outcome="execute", policy_applied="two_person_approve",
            execution_status="pending",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout
        )
    if rejected ≥ QUORUM_NEEDED:
        return FinalDecision(
            outcome="block", policy_applied="two_person_reject",
            execution_status="not_required",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout
        )
    if approved ≥ 1 and rejected ≥ 1:
        return FinalDecision(
            outcome="block", policy_applied="conservative_wins",
            execution_status="not_required",
            approved_count=approved, rejected_count=rejected, timeout_count=timeout
        )
    return None

async def build_final_decision_if_ready(
    r: redis.Redis, incident_id: str
) → Incident:
    incident = await _load_incident(r, incident_id)
    if incident.final_decision is not None:
        return incident

```

```

decision = _evaluate(incident)
if decision is None:
    return incident
incident.final_decision = decision
incident.state = (
    IncidentState.RESOLVED if decision.outcome == "block"
    else IncidentState.PENDING_EXECUTION
)
await _save_incident(r, incident)
return incident

```

Verificación

 VERIFICACIÓN

```
pytest soar/approval_api/tests/test_handlers.py -v
```

Esperado:

SALIDA ESPERADA

```

test_handlers.py::test_evaluate_decision_matrix[inputs0-execute-two_person_approve] PASSE
test_handlers.py::test_evaluate_decision_matrix[inputs1-block-two_person_reject] PASSE
test_handlers.py::test_evaluate_decision_matrix[inputs2-block-conservative_wins] PASSE
test_handlers.py::test_evaluate_decision_matrix[inputs3-block-conservative_wins] PASSE
test_handlers.py::test_single_response_no_decision PASSE
===== 5 passed in 0.04s =====

```

2.8 Consolidation window de 60 segundos

Contexto

Background task que dado un `incident_id`, espera 60 s. Si no hay quorum, marca pendientes como TIMEOUT y evalúa una vez más. Si tampoco hay decisión (nadie respondió), crea `block / no_quorum_timeout`.

`consolidation.py`



```

"""Ventana de consolidación T2 – 60 segundos."""

from __future__ import annotations
import asyncio, logging, time
import redis.asyncio as redis

from argos_contracts.enums import ApproverStatus, IncidentState
from argos_contracts.incident import FinalDecision
from soar.approval_api.handlers import _evaluate, _load_incident, _save_incident

logger = logging.getLogger(__name__)

WINDOW_SECONDS = 60

async def consolidation_task(r: redis.Redis, incident_id: str) → None:
    await asyncio.sleep(WINDOW_SECONDS)
    incident = await _load_incident(r, incident_id)
    if incident.final_decision is not None:
        return

    for ap in incident.approvers:
        if ap.status == ApproverStatus.PENDING:
            ap.status = ApproverStatus.TIMEOUT
            ap.responded_at = time.time()

    decision = _evaluate(incident)
    if decision is None:
        decision = FinalDecision(
            outcome="block", policy_applied="no_quorum_timeout",
            execution_status="not_required",
            approved_count=0, rejected_count=0,
            timeout_count=sum(1 for a in incident.approvers
                             if a.status == ApproverStatus.TIMEOUT),
        )
    incident.final_decision = decision
    incident.state = IncidentState.RESOLVED
    await _save_incident(r, incident)

```


Verificación

VERIFICACIÓN

```
pytest soar/approval_api/tests/test_consolidation.py -v
```

Esperado:

SALIDA ESPERADA

```
test_consolidation.py::test_consolidation_no_response_blocks PASSED
test_consolidation.py::test_consolidation_partial_quorum_blocks PASSED
===== 2 passed in 0.06s =====
```

Checklist Fase 2

#	Check	OK
1	<code>pytest soar/ -q</code> → ≥ 30 passed	☐
2	<code>uvicorn ... port 8001</code> arranca sin errores	☐
3	<code>/healthz</code> devuelve <code>{"ok": true, "redis": true}</code>	☐
4	Telegram, Discord smoke tests devuelven <code>success=True</code>	☐
5	Twilio dispara llamada real	☐
6	Tier router asigna T0 para <code>T1486</code> + <code>severity=LOW</code>	☐
7	Two-person resuelve conflict → <code>block / conservative_wins</code>	☐
8	Consolidation 60s cierra a <code>no_quorum_timeout</code> cuando nadie responde	☐

Fase 3 — Integración real

3.1 Consumer del stream `events:normalized`

Contexto

Hasta ahora procesabas `NormalizedEvent` fabricados a mano en tests. Ahora vas a leerlos de un Redis Stream alimentado por las 4 capas (P3 inyecta Sigma/Canary, P2 inyecta ML/LLM). El contrato está en `docs/contracts/CONTRACTS_SPECIFICATION.md`.

`consumer.py`



```

"""Consumer del stream events:normalized → emite Incident a Redis."""

from __future__ import annotations
import asyncio, logging, os, uuid
from datetime import datetime, timezone

import redis.asyncio as redis

from argos_contracts.enums import IncidentState, Tier
from argos_contracts.incident import HostInfo, Incident, NormalizedEvent
from soar.decision_engine.tier_router import route
from soar.notifications.service import NotificationService

logger = logging.getLogger(__name__)

STREAM = "events:normalized"
GROUP = "soar-router"
CONSUMER = os.environ.get("SOAR_CONSUMER_NAME", "soar-1")
INCIDENT_TTL_SECONDS = 6 * 3600

async def ensure_group(r: redis.Redis) → None:
    try:
        await r.xgroup_create(STREAM, GROUP, id="0", mkstream=True)
    except redis.ResponseError as e:
        if "BUSYGROUP" not in str(e):
            raise

async def _process_event(r: redis.Redis, notif: NotificationService, raw: dict) → Incident:
    event = NormalizedEvent.model_validate_json(raw["data"])
    tier = route(event)
    incident = Incident(
        incident_id=f"inc-{uuid.uuid4().hex[:12]}",
        tier=tier, state=IncidentState.NEW,
        severity=event.severity,
        host=HostInfo(hostname=event.host, ip="", os=""),
        mitre_technique=event.mitre_technique,
        num_layers_fired=event.num_layers_fired,
        confidence_score=event.confidence_score,
        created_at=datetime.now(timezone.utc),
        requires_approval=(tier == Tier.T2),
        approvers=[], final_decision=None, consolidation_window=None,
    )

```

```

await r.setex(
    f"incident:{incident.incident_id}",
    INCIDENT_TTL_SECONDS, incident.model_dump_json(),
)
results = notif.dispatch_for_tier(incident)
logger.info("incident %s → tier=%s notif=%s",
            incident.incident_id, tier.value,
            [(r.channel.value, r.success) for r in results])

async def run_consumer(notif: NotificationService) → None:
    r = redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
    await ensure_group(r)
    logger.info("consumer %s reading group %s on stream %s",
                CONSUMER, GROUP, STREAM)
    while True:
        resp = await r.xreadgroup(GROUP, CONSUMER, {STREAM: ">"}, count=10, block=500)
        for _stream, entries in resp or []:
            for entry_id, fields in entries:
                try:
                    await _process_event(r, notif, fields)
                    await r.xack(STREAM, GROUP, entry_id)
                except Exception: # noqa: BLE001
                    logger.exception("failed to process %s; entry retained", entry_id)

```

Comandos (arrancar consumer)


BASH

```
python -m soar.decision_engine.consumer
```

Salida esperada

SALIDA ESPERADA

```
INFO:soar.decision_engine.consumer:consumer soar-1 reading group soar-router on stream ev
```

Comandos (inyectar evento de prueba en otra terminal)


BASH

```
redis-cli XADD events:normalized '*' data \
  '{"event_id":"evt-001","severity":"CRITICAL","mitre_technique":"T1486","num_layers":1}'
```

Salida esperada (terminal del consumer)

SALIDA ESPERADA

```
INFO:soar.decision_engine.consumer:incident inc-abc123def456 → tier=T0 notif=[('telegram',
```

Verificación


VERIFICACIÓN

```
redis-cli KEYS 'incident:*'
redis-cli GET $(redis-cli KEYS 'incident:*' | tail -1) | jq '.tier, .host.hostname,
```

Esperado:

SALIDA ESPERADA

```
"incident:inc-abc123def456"
"T0"
"WIN-VICTIM-01"
"T1486"
```

Si algo falla

Síntoma	Causa	Fix
<code>BUSYGROUP Consumer</code> <code>Group name already exists</code> en logs	El grupo ya existía. No es error real.	<code>redis-cli XGROUP DESTROY events:normalized</code> <code>soar-router && redis-cli XGROUP CREATE</code> <code>events:normalized soar-router 0 MKSTREAM</code>
<code>Connection refused</code> <code>6379</code>	Redis no corre	<code>docker run -d -p 6379:6379 --name argos-</code> <code>redis redis:7</code>
Evento no genera incident	El consumer no consume del grupo	Verifica <code>redis-cli XINFO GROUPS</code> <code>events:normalized</code>

3.2 Audit log en PostgreSQL

Contexto

Cada `Incident` y cada respuesta de aprobador se persiste en PostgreSQL (P4 levantó la DB con `pgaudit` activo). Razón: trazabilidad forense post-mortem.

Schema (acordado con P4, ya en `lab/postgres/init.sql`)



SQL

```

CREATE TABLE IF NOT EXISTS audit_incidents (
    incident_id    text PRIMARY KEY,
    tier            text NOT NULL,
    severity       text NOT NULL,
    host           text NOT NULL,
    technique      text NOT NULL,
    created_at     timestampz NOT NULL,
    final_outcome  text,
    final_policy   text,
    final_at       timestampz,
    payload        jsonb NOT NULL
);

CREATE TABLE IF NOT EXISTS audit_responses (
    id              bigserial PRIMARY KEY,
    incident_id     text REFERENCES audit_incidents(incident_id),
    approver_id     text NOT NULL,
    channel         text NOT NULL,
    decision        text NOT NULL,
    received_at     timestampz NOT NULL
);

```

Verificación (después de correr un UC end-to-end)

VERIFICACIÓN

```

psql postgresql://argos:argos@localhost:5432/argos_audit -c \
    "SELECT incident_id, tier, technique, final_outcome FROM audit_incidents ORDER BY

```

Esperado:

SALIDA ESPERADA

incident_id	tier	technique	final_outcome
inc-abc123def456	T0	T1486	execute
inc-987654321cba	T2	T1083	block

(2 rows)

3.3 Hook al LLM Triage (Layer 4 de P2)

Contexto

Cuando el `tier_router` asigna T2, invocas el LLM Triage de P2 que devuelve `LLMVerdict` con label/confidence/reasoning. Enriquece el `Incident` antes de notificar.

Patrón de integración

 PYTHON

```
# soar/decision_engine/consumer.py (modificación dentro de _process_event)

if tier == Tier.T2:
    from ml.llm_triage import classify as llm_classify
    try:
        verdict = await llm_classify(event)
        incident.llm_verdict = verdict
    except Exception: # noqa: BLE001
        logger.exception("llm classify failed; continuing without enrichment")
        incident.llm_verdict = None
```

Importante: si el LLM falla (rate-limit, timeout, OpenAI caído), el incidente sigue su curso sin enriquecimiento. **Nunca bloquee** el flujo HITL por el LLM.

Verificación

VERIFICACIÓN

```
redis-cli GET $(redis-cli KEYS 'incident:*' | tail -1) | jq '.llm_verdict'
```

Esperado (cuando el evento fue T2):

SALIDA ESPERADA

```
{
  "label": "malicious",
  "confidence": 0.93,
  "reasoning": "...",
  "backend": "openai_gpt4o_mini",
  "latency_ms": 687
}
```

O `null` si el LLM falló (acceptable).

3.4 End-to-end UC-01 (ransomware T0)

Contexto

Validación viva del flujo completo: ataque → Sigma + ML + Canary disparan → SOAR asigna T0 → notificaciones → Wazuh active-response aísla el host. Todo en menos de 15 s.

Pasos manuales

Necesitas 3 terminales abiertas (idealmente con tmux/screen).

1. **Terminal 1** — consumer SOAR.
2. **Terminal 2** — Approval API (no se usa para T0 pero la dejas corriendo).
3. **Terminal 3** — vagrant ssh a la víctima donde corre el script de ataque.

Comandos

 BASH

```
# Terminal 1
python -m soar.decision_engine.consumer

# Terminal 2
uvicorn soar.approval_api.main:app --port 8001

# Terminal 3
vagrant ssh linux-victim
python /vagrant/attack-simulation/ransomware_simulator/lockbit_like.py \
    --variant uc01 --target linux-victim
```

Salida esperada

En Terminal 3:

SALIDA ESPERADA

```
[uc01] generated key: gAAAAAB...
[uc01] encrypted 200 files
[uc01] (simulated) vssadmin delete shadows /all /quiet
```

En Terminal 1:

SALIDA ESPERADA

```
INFO:soar.decision_engine.consumer:incident inc-XYZ789ABC123 → tier=T0 notif=[('telegram'
```

En tu Telegram, dentro de 10 s:

SALIDA ESPERADA

❑ ARGOS T0 – linux-victim
Técnica: T1486
Capas firing: 3
Confianza: 0.94
ID: inc-XYZ789ABC123

Verificación

VERIFICACIÓN

```
# Latencia
psql postgresql://argos:argos@localhost:5432/argos_audit -c \
  "SELECT incident_id, EXTRACT(EPOCH FROM (final_at - created_at)) AS seconds_to_resolve
  FROM audit_incidents WHERE technique='T1486' ORDER BY created_at DESC LIMIT 1;"
```

Esperado:

SALIDA ESPERADA

incident_id	seconds_to_resolve
inc-XYZ789ABC123	8.42

Tiempo < 15 s = .

3.5 End-to-end UC-04 (two-person rule)

Contexto

UC-04 es el centerpiece del demo. Un T2 dispara, los 4 integrantes reciben Telegram con botones, se espera quorum de 2 approves o conflict.

Comando (en Terminal 3, vagrant ssh)



BASH

```
python /vagrant/attack-simulation/ransomware_simulator/postgres_attack.py \  
--target linux-victim
```

Timeline esperado

SALIDA ESPERADA

```
t=0s      Ataque lanzado. Sigma + ML disparan.  
t=2s      SOAR crea Incident T2 → notifica 4 integrantes.  
t=10s     P1 toca [Approve].  
t=18s     P2 toca [Approve] → quorum 2 → final_decision=execute → Wazuh aísla.  
          Streamlit muestra banner verde: "2 approve · 0 reject · execute".
```

Variante conflict

SALIDA ESPERADA

```
t=10s     P1 toca [Approve]  
t=18s     P2 toca [Reject]  
          → conservative-wins → block / conservative_wins  
          → Streamlit banner gris: "1 approve · 1 reject · block".
```

Variante timeout

SALIDA ESPERADA

```
t=10s     P1 toca [Approve]  
t=55s     Twilio llama a P3 y P4.  
t=60s     Window cierra → no quorum → block / no_quorum_timeout.
```

Verificación

VERIFICACIÓN

```
redis-cli GET $(redis-cli KEYS 'incident:*' | tail -1) | jq '.final_decision'
```

Esperado (caso aprobado):

SALIDA ESPERADA

```
{
  "outcome": "execute",
  "policy_applied": "two_person_approve",
  "execution_status": "success",
  "approved_count": 2,
  "rejected_count": 0,
  "timeout_count": 2
}
```

Si algo falla

Síntoma	Causa	Fix
Botones no aparecen en Telegram	<code>requires_approval=False</code>	Verifica que el incident T2 tiene <code>requires_approval=True</code> (Tier.T2 lo setea en el consumer)
Aprobar no actualiza Redis	Telegram callback no llega	El webhook de Telegram apunta a tu Approval API; usa <code>ngrok</code> y registra: <code>curl https://api.telegram.org/bot\$TOKEN/setWebhook?url=https://abcd.ngrok.io/telegram/callback</code>
Streamlit no refresca	<code>streamlit-autorefresh</code> desactivado	F5 manual; los datos están en Redis

Checklist Fase 3

#	Check	OK
1	Consumer SOAR consume del stream real	☐
2	UC-01 corre end-to-end en < 15 s	☐
3	UC-04 con 2 approves → execute	☐
4	UC-04 conflict → block / conservative_wins	☐
5	UC-04 timeout → block / no_quorum_timeout	☐
6	<code>audit_incidents</code> se llena con cada UC	☐
7	LLM hook degrada graciosamente si OpenAI cae	☐
8	<code>pytest -q</code> global ≥ 80 passed	☐

Fase 4 — Rehearsal y polish

4.1 Rehearsal individual (silencioso)

Contexto

Antes de invitar al equipo, tú haces el corrido completo solo. Mides latencias y registras fallos.

Comandos (con cronómetro)



BASH

```
make demo-up

time {
  python attack-simulation/ransomware_simulator/lockbit_like.py --variant uc01 --target linux-victim
  sleep 20
  python attack-simulation/ransomware_simulator/canary_path.py --target linux-victim
  sleep 15
  python attack-simulation/ransomware_simulator/postgres_attack.py --target linux-victim
}
```

Verificación — registrar métricas

VERIFICACIÓN

```
psql postgresql://argos:argos@localhost:5432/argos_audit -c \
"SELECT technique, tier, \
      EXTRACT(EPOCH FROM (final_at - created_at)) AS sec_to_resolve, \
      final_outcome, final_policy \
FROM audit_incidents ORDER BY created_at DESC LIMIT 5;"
```

Esperado:

SALIDA ESPERADA

technique	tier	sec_to_resolve	final_outcome	final_policy
T1486	T0	7.12	execute	auto_t0
T1083	T0	5.84	execute	auto_t0
T1190	T2	22.45	execute	two_person_approve

(3 rows)

4.2 Rehearsal en grupo

Pasos manuales

1. Crear evento Discord call: "ARGOS Rehearsal Final" para una hora acordada.
2. Cada integrante con su celular cargado, Telegram + Discord abiertos.
3. P4 confirma que su lab está arriba; tú confirmas lab espejo arriba.
4. Compartes pantalla con Streamlit Console (tab 2 Approval Console).
5. Sigues el script de 12-13 minutos del Manual de Equipo §2.2.

Verificación

Check	Esperado
Ningún integrante reporta no haber recibido Telegram	✓
Streamlit refresca ≤ 2 s tras cada acción	✓
Twilio entra a celular acordado	✓
Audit log final tiene los 3 escenarios UC-04 distinguibles	✓
Duración total ≤ 13 min	✓

4.3 Edge cases y contingencias

Tabla de fallos previstos

Fallo	Probabilidad	Respuesta inmediata
WiFi del salón bloquea <code>api.openai.com</code>	Alta	<code>export LLM_BACKEND=llama_local && make llm-restart</code>
Twilio trial sin saldo	Media	Verificar saldo T-24 h. Si < USD 3, top-up con USD 5
Wazuh manager no arranca	Media	Switch a lab espejo: <code>export WAZUH_HOST=p1-espejo.local && make soar-restart</code>
Postgres lleno > 90 %	Baja	<code>make audit-vacuum</code>
Crash del consumer SOAR mid-demo	Media	<code>make soar-restart</code> (reinicia en < 5 s)
Streamlit no actualiza	Media	F5 manual
Llamada Twilio se va a buzón	Alta	Por diseño: <code>no_quorum_timeout</code> aplica

Plan de fallback nuclear

Si tres fallos consecutivos durante la primera parte del demo:

1. Detener intentos en vivo.
2. Mostrar video pre-grabado (P4 lo grabó en rehearsal).
3. P1 narra sobre el video.

Tener el video listo es no-negociable.

4.4 Checklist pre-demo (T-2 h)

#	Check	OK
1	Lab primario (P4) — <code>vagrant status</code> todos UP	☐
2	Lab espejo (P1) — <code>vagrant status</code> todos UP	☐
3	Telegram bot envía mensaje de prueba a los 4 celulares	☐
4	Discord webhook envía mensaje al canal	☐
5	Twilio: credit > USD 3	☐
6	OpenAI: usage budget no excedido	☐
7	Postgres responde a <code>psql -c "\dt"</code>	☐
8	OpenSearch dashboard carga en <code>http://localhost:5601</code>	☐
9	Streamlit Console abre en <code>http://localhost:8501</code>	☐
10	Video respaldo grabado y accesible offline	☐
11	Pendrive USB con repo clonado	☐
12	Cargadores de las 2 laptops + cable al proyector	☐

Checklist Fase 4

#	Check	OK
1	Rehearsal individual cerrado sin issues abiertos	☐
2	Rehearsal en grupo cerrado con los 4 integrantes	☐
3	Video respaldo grabado	☐
4	Checklist pre-demo corrido T-2 h	☐
5	<code>docs/LESSONS_LEARNED.md</code> actualizado	☐

Apéndice A — Troubleshooting consolidado

#	Síntoma	Diagnóstico rápido	Fix
A. 1	<code>ModuleNotFoundError: argos_contracts</code>	<code>which python</code> no es venv	<code>source .venv/bin/activate && pip install -e ./argos_contracts</code>
A. 2	Telegram <code>"chat not found"</code>	Nunca enviaste mensaje al bot	<code>t.me/<usuario_bot></code> → <code>/start</code>
A. 3	Discord webhook 401	URL mal copiada	Recrea el webhook en el canal
A. 4	Twilio error 21219	<code>TO_NUMBER</code> no verificado	Verifícalo en console.twilio.com
A. 5	Redis <code>CONNECTION REFUSED</code>	Daemon parado	<code>docker run -d -p 6379:6379 redis:7</code>
A. 6	<code>BUSYGROUP</code> en logs	Grupo ya existe	Ignorar — no es error
A. 7	Approval API <code>Event loop is closed</code>	Mal asyncio fixture	<code>pip install pytest-asyncio==0.23.x</code> y <code>asyncio_mode = "auto"</code> en <code>pyproject.toml</code>
A. 8	Tests <code>xfail</code> que pasan (<code>XPASSED</code>)	Bug resuelto	Borrar el mark <code>@pytest.mark.xfail</code>
A. 9	Latencias notif > 2 s	DNS o red lenta	<code>time curl https://api.telegram.org/bot\$TOKEN/getMe</code> — si > 800 ms, cambiar red

Apéndice B — Comandos de emergencia (durante el demo)

BASH

```
# Reiniciar todo el plano de control SOAR sin tocar el lab
make soar-restart

# Switch a Llama local si OpenAI cae
export LLM_BACKEND=llama_local && make llm-restart

# Switch al lab espejo
export WAZUH_HOST=p1-espejo.local && make soar-restart

# Limpiar incidents del demo previo
redis-cli --scan --pattern 'incident:*' | xargs -r redis-cli DEL

# Resetear consumer group (re-procesa últimos eventos)
redis-cli XGROUP SETID events:normalized soar-router 0

# Forzar block manual de un incident colgado
redis-cli SET 'incident:inc-XXX:override' 'force_block'
```

Apéndice C — Referencias cruzadas

Cuando estés en...	Lee
<code>tier_router.py</code>	SAD §6.2, ADR-0002
<code>notifications/</code>	ADR-0007 v2, SAD §8.3
<code>approval_api/</code>	ADR-0006, USE_CASES UC-04
<code>consumer.py</code>	<code>docs/contracts/CONTRACTS_SPECIFICATION.md</code> §4
<code>audit/</code>	THREAT_MODEL §5, SAD §11
Contingencias del demo	<code>docs/team/manual-equipo.md</code> §1, THREAT_MODEL §7

Change log

Versión	Fecha	Cambio
3.0	2026-05-24	Reestructurado a Contexto → Pasos manuales → Comandos → Salida esperada → Verificación → Si algo falla. Bloques de comandos preparados para HTML interactivo con copy buttons. Eliminadas referencias temporales (semanas/días/sprint). Renombrado de <code>sprint-week-1-p1-enzo.md</code> a <code>manual-p1-enzo.md</code> .